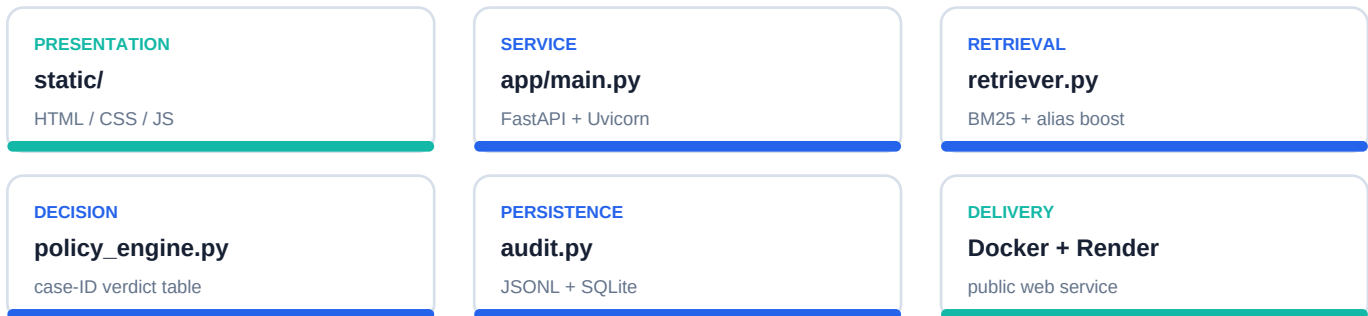


System Architecture

1 Policy-grounded internal IT decision service

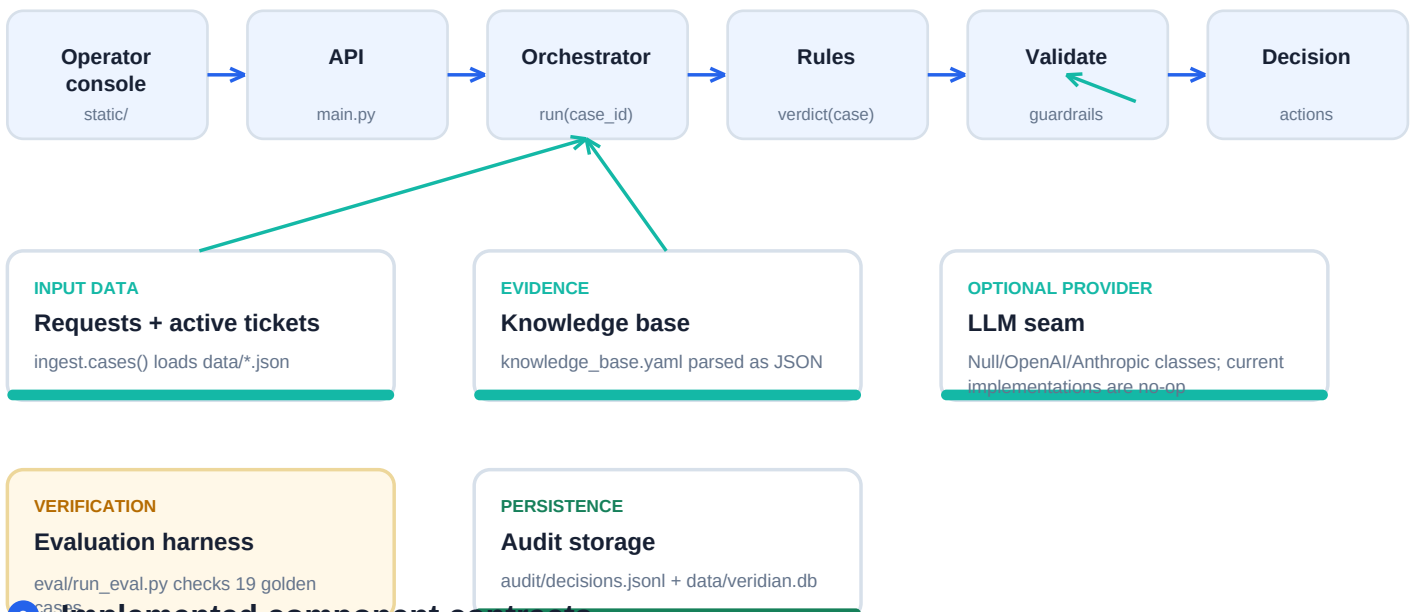
FastAPI + local data + deterministic rules

The service reads fixed employee requests and active tickets, retrieves local policy evidence, produces a Pydantic Decision from deterministic case rules, validates it, and records the result. Its UI is a vanilla single-page console served by FastAPI.



2 High-level runtime architecture

Figure 1



3 Implemented component contracts

MODULE	INPUT / FUNCTION	OUTPUT
app/main.py	HTTP routes; StaticFiles mount	JSON API + operator console
app/orchestrator.py	Orchestrator.run(case_id)	retrieved chunks -> final Decision
app/retriever.py	Retriever.search(request)	top-k KB chunks with score
app/policy_engine.py	verdict(case)	preliminary Pydantic Decision
app/audit.py	record(case, chunks, decision)	JSONL append + SQLite upsert

Implementation note: classifier.py, precedent.py, and conflict_detector.py exist but are not invoked by Orchestrator.run. Their roles are not represented as runtime nodes above.

1 Processing sequence

Figure 2



2 Phase-by-phase contract

Purpose | input | processing | output

PHASE	INPUT	PROCESSING	OUTPUT
1. Case input	case_id	ingest.cases reads 15 requests plus active tickets	case dict
2. Retrieval	case request	BM25 over KB title/text/aliases; alias bonus	chunk list + score
3. Policy verdict	case dict	case-ID rule table; sla.py computes dates/age	Decision model
4. Optional LLM	Decision + chunks	provider selected by LLM_PROVIDER; current providers return unchanged	same Decision
5. Guardrails	Decision	uncited route; low confidence downgrade; conflict route	final Decision
6. Action + audit	final Decision	action labels returned; record writes persistence	JSONL and SQLite

3 Design-only helpers not on the live path

Marked to avoid overstating implementation

HELPER NORMALIZE

No call in Orchestrator.run.

HELPER CLASSIFY

No call in Orchestrator.run.

HELPER PRECEDENT

No call in Orchestrator.run.

HELPER CONFLICT DETECTOR

No call in Orchestrator.run.

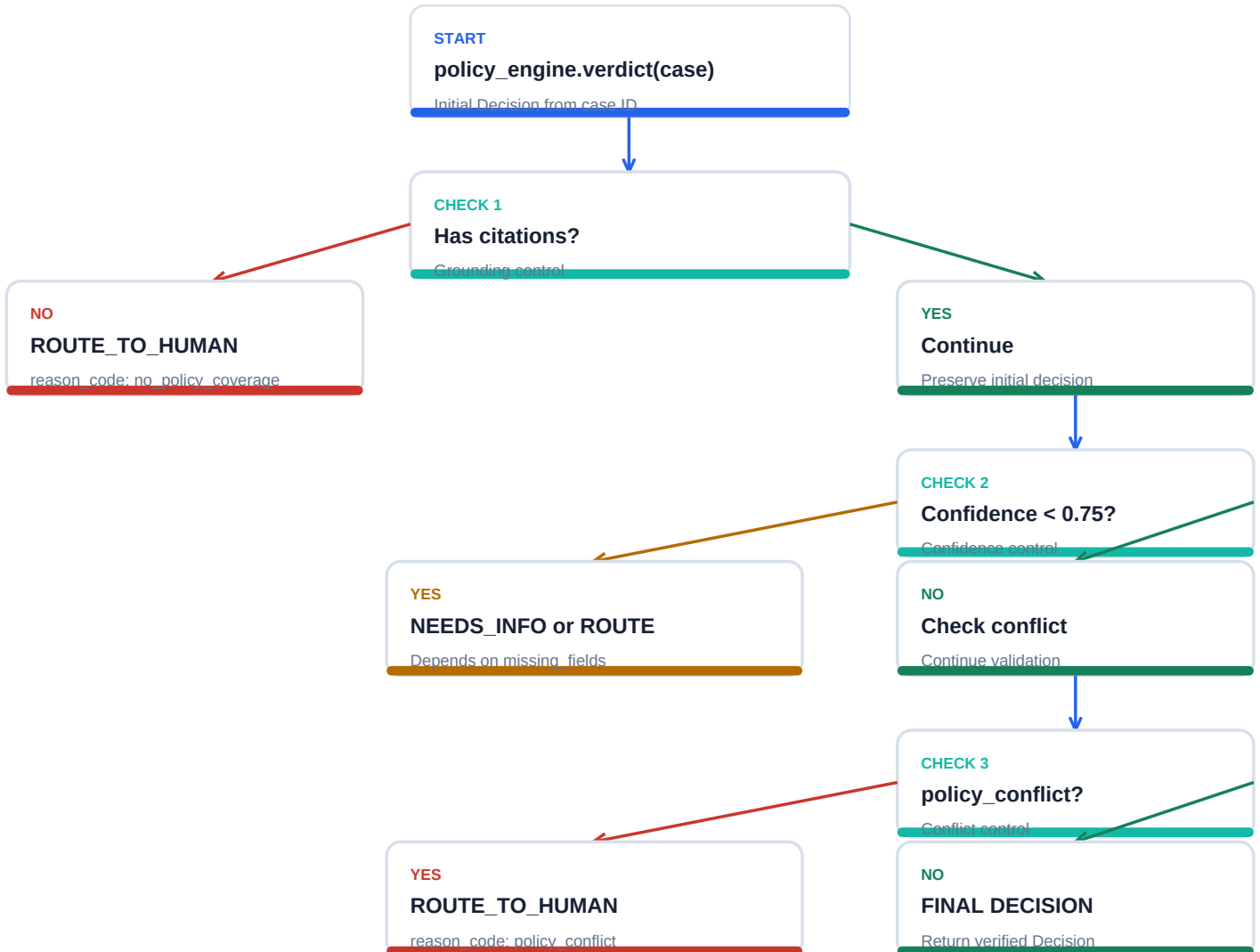
The actual pipeline is therefore retrieve -> deterministic verdict -> enrichment -> validation -> proposed action -> audit. This distinction is based on app/orchestrator.py.

Decision, Evidence and Guardrails

Veridian Service Agent | Detailed Architecture

1 Decision validation tree

Figure 3 - actual guardrails.validate behavior

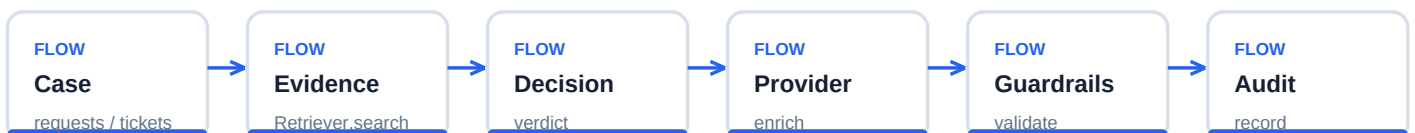


2 Outcome semantics from policy_engine.verdict

OUTCOME	WHEN THE IMPLEMENTED TABLE USES IT
AUTO_RESOLVE	Known covered instruction or queue action: Wi-Fi, unlock, VPN renewal, archive, repair, review queue.
NEEDS_INFO	Required evidence, approval, or account state missing; REQ-15 is downgraded for 0.42 confidence.
ROUTE_TO_HUMAN	Policy conflict, approval boundary, or no policy coverage, such as REQ-01 and REQ-10.
FLAG_RISK	Active security violation; REQ-08 produces containment actions.
MONITOR	Active work is already with the correct owner and within SLA / fulfilment state.

3 Evidence and data flow

Figure 4



audit.record stores input, retrieved chunk IDs, reasoning trace, final Decision, and timestamp in append-only JSONL; it also upserts SQLite decisions(case_id, payload, created_at). Authority is encoded in verdict rules and owner values - not a separate guardrail function.

Implementation, API and Real Case

Veridian Service Agent | Detailed Architecture

1 Repository structure

Important files only

PATH	RESPONSIBILITY
app/	FastAPI, Decision schema, orchestration, local retrieval, hard-coded policy rules, guardrails, audit, provider seam.
data/	requests.json, tickets.json, knowledge_base.yaml parsed as JSON, golden_set.json, runtime SQLite database.
static/	Single-page operator console: index.html, app.js, styles.css.
eval/ + tests/	No-LLM 19-case golden evaluation and pytest API/engine tests.
Dockerfile + render.yaml	Container execution and Render service configuration.

2 HTTP API

app/main.py

METHOD	ENDPOINT	PURPOSE
GET	/healthz	Health, frozen clock, deterministic mode.
GET	/api/cases and /api/kb	List case inputs or stored policy records.
GET	/api/cases/{id}	Run one case and return Decision; 404 if absent.
POST	/api/run and /api/run/{id}	Run every case or one specified case.
POST	/api/ask	KB-only answer; retrieval score < 3 returns not covered.
GET	/api/metrics and /api/audit	Calculated decision metrics; last 100 JSONL audit records.

3 Real case walkthrough: REQ-08 phishing forwarding

Input -> evidence -> rule -> guardrail -> action -> audit

INPUT Input Suspected phishing is being forwarded to teammates.	EVIDENCE Evidence KB-09 and active precedent TK-1048.	VERDICT Verdict FLAG_RISK; owner IT Security; confidence 0.99.
VALIDATE Validate Citations present; no confidence or conflict override.	ACTION Action Issue containment instruction; preserve Security escalation.	AUDIT Audit Record case, retrieved IDs, trace, and final Decision.

4 Why the final decision is safe

Implemented behavior

policy_engine.verdict identifies forwarding as a KB-09 security violation and drafts containment instructions: stop forwarding, do not click links, recall/delete copies, and report Security. actions.execute only returns the action labels; it makes no external system call. audit.record preserves the final FLAG_RISK decision.

DEPLOYMENT

Docker runs evaluation and batch processing before Uvicorn. Render runs app.main:app on \$PORT and health-checks /healthz. render.yaml sets LLM_PROVIDER=null.